Félix Arnold

ECE 6100 - Project 2
Tomasulo & Branch Predictor Simulator

Now that the simulator works properly, I need to find the optimum pipeline configuration for each trace. To this end we must get best possible Instructions Per Cycle with the lowest resource utilization.

deepsjeng531

We are going to start with the deepsjeng531 trace. The first parameters we need to set up is which branch predictor is the best match. To this end, we are going to simulate a few identical configurations using both predictors to compare their output. Here are the results:

	Conf 1	Conf 2	Conf 3
Num of RS per FU	4	6	8
Num of ALU	1	3	4
Num of MUL	1	2	3
Num of LSU	1	2	2
Fetch Width	2	4	8

IPC with Yeh-Patt	Num (log) of entries in pattern table: 14	0.946	1.528	1.657
	Num (log) of entries in pattern table: 15	0.948	1.536	1.676
IPC with GShare	Num (log) of 2-bits counter: 15	0.91	1.395	1.472
	Num (log) of 2-bits counter: 16	0.915	1.401	1.472

Thanks to this, we can observe that the Yeh-Patt branch predictor have better performance for this trace than the GShare. Moreover, as expected, we can see that the more resources we allocate, the higher the IPC. We can get at most 1.676 instructions per cycle, so our final configuration must have an IPC of at least 1.508, as we want to perform at least 90% as well as the best performer.

We want to minimize the resources, which means we are looking for the smallest and least power-consuming configuration. We already know we should not further study with the GShare predictor as the resulted IPC are too low. As we observe a very small performance difference between both configuration of the Yeh-Patt predictor we keep use the one using the least amount of space.

With $p = 14$, $H = 11$, the predictor use:

$2^H \times 14 = 2^{11} \times 14 = 28672$ bits for the pattern table

$2^P \times 2 = 2^{14} \times 2 = 32768$ bits for the history table

While with $p = 15$, $H = 12$, it uses:

$2^H \times P = 2^{12} \times 15 = 61440$ bits for the pattern table

$2^P \times 2 = 2^{15} \times 2 = 65536$ bits for the history table

We are now going to going to study the number of function units. We set the number of reservation station per function unit to its maximum value, and the same goes for the fetch width. After several simulations, we observe that the minimum amount of FU to still reach an IPC of 1.508 is 2 ALU, 1 MUL and 2 LSU – we reach an IPC of 1.511 in that case. Having less than this amount for each different FU greatly undermines the pipeline efficiency.

However, we also observe that decreasing the number of reservation station has a minor impact on the instruction per cycle. For example, with $s = 7$, we get $IPC = 1.510$. However, below this value, we need to increase the number of ALU up to 3 ALU, 2 MUL and 2 LSU to reach good performance. We also see that using $P = 15$ and $H = 12$, we can reduce the number of RS per FU up to 5 without increasing the number of function units.

We can also study the impact of fetch width and observe that we need at least to be able to fetch at least 4 instruction per cycle to reach good performance. In that case, we also need at least 3 ALU, 2 MUL and 2 LSU and we limit the number of RS to 4.

This leaves us with choosing between these 3 configurations:

Num (log) of entries in pattern table:	Num (log) of entries in history table:	Num of RS per FU	Num of ALU	Num of MUL	Num of LSU	Fetch Width
14	11	7	2	1	2	8
15	12	5	2	1	2	8
14	11	4	3	2	2	4

Using a processor pipeline with less function unit would be more resource effective as more FU would much more increase the power consumption and the silicon area, compared to increasing the size of the queues. The first configuration in the table would require $7 \times (2 + 1 + 2) = 35$ reservation stations compared to 25 for the second one. Even for a real Tomasulo algorithm, a reservation station would be less than 400 bits wide (which is still a high margin). So, we save more space by having smaller predictor tables.

For the deepsjeng531 trace, we choose the following configuration, which achieves an IPC of 1.510:

- Yeh-Patt branch predictor with a pattern table of size 2^{14} and a history table of size 2^{11}
- 7 reservation station per function unit
- 2 ALU, 1 MUL, 2 LSU
- 8 instructions fetched at a time.

leela541

The same way as previously, we can start our study by the following simulation to determine the best branch predictor and the IPC value of the best performer.

	Conf 1	Conf 2	Conf 3
Num of RS per FU	4	6	8
Num of ALU	1	3	4
Num of MUL	1	2	3
Num of LSU	1	2	2
Fetch Width	2	4	8

IPC with Yeh-Patt	Num (log) of entries in pattern table: 14	1.074	1.830	2.068
	Num (log) of entries in pattern table: 15	1.074	1.826	2.064
IPC with GShare	Num (log) of 2-bits counter: 15	1.031	1.637	1.769
	Num (log) of 2-bits counter: 16	1.030	1.631	1.758

The best performer is obtained with the Yeh-Patt predictor and has an IPC value of 2.068, which means we want a configuration reaching at least 1.861. Thus, GShare is not suitable for this trace either.

We can also observe that for a fetch width of 4, we are limited to an IPC of 1.851, which is inferior to our minimum expected performance. Thus, we can look for the configuration that minimize the number of reservation station per FU and the number of FU.

Thanks to the simulations, we find that we only get a sufficient IPC by respecting the instructions when we use 3 ALU, 2 MUL and 2 LSU. For this amount of FU, 4 RS per FU is enough to get good performances. The same way, we can use a yeh-patt branch predictor with the configuration (14, 15).

For the leela541 trace, we choose the following configuration, which achieves an IPC of 1.974:

- Yeh-Patt branch predictor with a pattern table of size 2^{14} and a history table of size 2^{11}
- 4 reservation station per function unit
- 3 ALU, 2 MUL, 2 LSU
- 8 instructions fetched at a time.

mcf505

After several simulations, we found out that the one with the best performance is the following one:

- Yeh-Patt branch predictor with a pattern table of size 2^{15} and a history table of size 2^{12}
- 4 reservation station per function unit
- 2 ALU, 1 MUL, 2 LSU
- 8 instructions fetched at a time.

Its IPC value is 1.042, which means we want a configuration reaching at least 0.938. We observe that the GShare predictor does not perform well enough to reach this value, as we get at most 0.873.

We are not able to decrease the number of function units in use without impacting too much the performance of the pipeline. However, we are able to decrease the fetch width up to 4 and the table size of the branch predictor.

For the mcf505 trace, we choose the following configuration, which achieves an IPC of 0.991:

- Yeh-Patt branch predictor with a pattern table of size 2^{14} and a history table of size 2^{11}
- 4 reservation station per function unit
- 2 ALU, 1 MUL, 2 LSU
- 4 instructions fetched at a time.

nab544

The same way as previously, we can start our study by the following simulation to determine the best branch predictor and the IPC value of the best performer.

	Conf 1	Conf 2	Conf 3
Num of RS per FU	4	6	8
Num of ALU	1	3	4
Num of MUL	1	2	3
Num of LSU	1	2	2
Fetch Width	2	4	8

IPC with Yeh-Patt	Num (log) of entries in pattern table: 14	0.932	1.414	1.485
	Num (log) of entries in pattern table: 15	0.932	1.409	1.487
IPC with GShare	Num (log) of 2-bits counter: 15	0.885	1.288	1.342
	Num (log) of 2-bits counter: 16	0.887	1.297	1.352

The best performer is obtained with the Yeh-Patt predictor and has an IPC value of 1.487, which means we want a configuration reaching at least 1.338. We also need to consider GShare this time as it managed to reach these values.

For the Yeh-Patt branch predictor, the best resource-efficient configuration is the following one, with an IPC of 1.346.

Num (log) of entries in pattern table:	Num (log) of entries in history table:	Num of RS per FU	Num of ALU	Num of MUL	Num of LSU	Fetch Width
14	11	4	2	1	2	4

For the GShare branch predictor, the best resource-efficient configuration are the following ones.

Num (log) of entries in pattern table:	Num of RS per FU	Num of ALU	Num of MUL	Num of LSU	Fetch Width	IPC
16	5	3	2	2	8	1.338
15	4	4	2	2	8	1.342

In term of queue storage space, as seen previously, the Yeh-Patt would require 28672 bits for the pattern table and 32768 bits for the history table. Opposed to that, the GShare predictor would use $2^P \times 2 + P$ bits, meaning 65551 bits for $P = 15$ or 131088 bits for $P = 16$. The Yeh-Patt configuration is the one requiring the least amount of queue size for its scheduling queue. It is also the one using the smaller reorder buffer and register file, as well as the least amount of function units.

For the nab544 trace, we choose the following configuration, which achieves an IPC of 1.346:

- Yeh-Patt branch predictor with a pattern table of size 2^{14} and a history table of size 2^{11}
- 4 reservation station per function unit
- 2 ALU, 1 MUL, 2 LSU
- 4 instructions fetched at a time.

xz557

The same way as previously, we can start our study by the following simulation to determine the best branch predictor and the IPC value of the best performer.

	Conf 1	Conf 2	Conf 3
Num of RS per FU	4	6	8
Num of ALU	1	3	4
Num of MUL	1	2	3
Num of LSU	1	2	2
Fetch Width	2	4	8

IPC with Yeh-Patt	Num (log) of entries in pattern table: 14	1.030	1.064	1.067
	Num (log) of entries in pattern table: 15	1.030	1.064	1.067
IPC with GShare	Num (log) of 2-bits counter: 15	1.029	1.061	1.063
	Num (log) of 2-bits counter: 16	1.029	1.061	1.063

The best performer is obtained with the Yeh-Patt predictor and has an IPC value of 1.067, which means we want a configuration reaching at least 0.960. We observe that all possible configurations match this requirement and we do not gain any significant performance boost when allocating more resources.

The only choice we need to do is which branch predictor to use. As seen previously, the Yeh-Patt would use 61440 bits in total and the GShare 65551 bits. As such, we would prefer the Yeh-Patt one.

For the leela541 trace, we choose the following configuration, which achieves an IPC of 1.030:

- Yeh-Patt branch predictor with a pattern table of size 2^{14} and a history table of size 2^{11}
- 4 reservation station per function unit
- 1 ALU, 1 MUL, 1 LSU
- 2 instructions fetched at a time.